TP 2 : Langage C

Dre.khawla Hamouichou

Résumé du langage C :

Le **langage C** est un **langage de programmation généraliste**, créé au début des années 1970 (mille neuf cent soixante-dix) par **Dennis Ritchie** aux laboratoires **Bell Labs**.

Il est l'un des langages les plus influents au monde — de nombreux langages modernes (C++, Java, Python…) en dérivent.

Caractéristiques principales :

1. **Langage structuré :**
   → Les programmes sont organisés en fonctions, ce qui facilite la lecture et la maintenance du code.
2. **Langage compilé :**
   → Le code source (.c) est traduit en langage machine grâce à un **compilateur** (ex : GCC, Dev-C++).
3. **Langage rapide et efficace :**
   → Permet de manipuler directement la mémoire et les adresses (via les pointeurs).
4. **Langage portable :**
   → Un même programme peut être exécuté sur différents systèmes (Windows, Linux, macOS) avec peu de modifications.
5. **Langage de bas niveau et de haut niveau :**
   → Il permet à la fois la manipulation proche du matériel (comme l'assembleur) et la création d'algorithmes complexes (haut niveau).

Structure d'un programme en C :

```c
#include <stdio.h>

int main() {
    printf("Bonjour tout le monde !\n");
    return 0;
}
```

- #include <stdio.h> : indique qu'on utilise une **bibliothèque standard** pour les entrées/sorties **(pour printf)** .
- int main() : **point d'entrée** du programme.
- printf() : affiche un texte à l'écran.
- return 0; : indique que le programme s'est terminé correctement.

Les éléments de base du langage C :

Élément	Description	Exemple
Variables	Espaces mémoire pour stocker des valeurs	int age = 20;
Types de données	int, float, char, double…	float note = 17.5;
Opérateurs	+, -, *, /, %…	somme = a + b;
Conditions	if, else, switch	if (x > 0) printf("Positif");
Boucles	for, while, do...while	for(i=0;i<10;i++)
Fonctions	Blocs de code réutilisables	int somme(int a, int b)
Tableaux	Ensemble de valeurs du même type	int tab[5];
Pointeurs	Variables contenant des adresses mémoire	int *p;

Avantages du langage C :

✅ Très rapide en exécution
✅ Langage idéal pour apprendre la programmation structurée
✅ Utilisé pour développer des systèmes d'exploitation, des pilotes, et des logiciels embarqués

Inconvénients :

⚠ Manipulation de la mémoire parfois complexe
⚠ Peu de protections contre les erreurs de programmation (ex : dépassement de tableau)
⚠ Syntaxe stricte pour les débutants

En résumé :

Le langage C est **puissant, rapide et fondamental**.
Il constitue la **base de la programmation moderne**, indispensable pour comprendre les principes de la mémoire, de la compilation et de la logique algorithmique.

Lancement & présentation de Dev-C++ :

Objectif :

Faire découvrir l'environnement de développement aux étudiants : à quoi sert Dev-C++, comment créer un fichier, écrire, compiler et exécuter un programme.

- Dev-C++ est un logiciel qu'on utilise pour écrire, traduire et exécuter nos programmes en langage C.
  Il contient un éditeur de texte **(Pour écrire le code source.)**, un compilateur**(Pour traduire le code source)** et un terminal **(console d'exécution).**

1. **Ouvrir Dev-C++**
   - Cliquer sur le menu Démarrer → Rechercher "Dev-C++" => c'est un IDE (Integrated Development Environment).
2. **Créer un nouveau fichier source :**
   - Menu Fichier → Nouveau → Fichier Source => Expliquer que c'est dans ce fichier qu'on écrit le **code C**.
3. **Sauvegarder le fichier :**
   - Menu Fichier → Enregistrer sous…
   - Nommer le fichier par exemple bonjour.c
     ☞ Insister sur **l'extension .c** (obligatoire pour un programme en langage C).
4. **Montrer la structure d'un programme simple :**

```c
#include <stdio.h>
int main() {
    printf("Bonjour tout le monde !");
    return 0;
}
```

Compilation et exécution d'un programme :

Objectif :

Faire comprendre la différence entre **écriture**, **compilation**, et **exécution**.

- Quand on écrit un programme en C, l'ordinateur ne le comprend pas directement.
  Il faut le **traduire en langage machine** grâce à un **compilateur**.

Ce que tu montres :

1. Cliquer sur **Exécuter** → **Compiler et exécuter** ou utiliser le raccourci F11.
2. Montrer que si le code est correct, la **console noire s'ouvre** et affiche le message.
3. Montrer aussi **ce qui se passe si on fait une erreur** :
   - Exemple : enlever un point-virgule ou une parenthèse.
   - Le compilateur affiche une **erreur en bas** avec un message.
   - Cliquer sur le message → Dev-C++ te montre la ligne exacte.
   - Expliquer :
     - **Erreur** → problème de syntaxe (mauvaise écriture).
     - **Avertissement (warning)** → le programme marche mais il y a un risque ou une mauvaise pratique.
4. Le compilateur fait deux étapes :
   - **Compilation :** vérifie la syntaxe et traduit le code en langage machine.
   - **Édition de liens (linking) :** assemble ton code avec les bibliothèques (par exemple stdio.h pour printf).

Empêcher la fermeture de la console d'exécution :

Objectif :

Montrer pourquoi la fenêtre se ferme trop vite et comment y remédier.

✓ Par défaut, après avoir exécuté un programme, la fenêtre se ferme immédiatement. Pour avoir le temps de lire le résultat, on peut ajouter une commande spéciale à la fin du programme.

1. Sans la commande :

```c
printf("Bonjour tout le monde !");
```

➜ La console s'ouvre et se ferme immédiatement.

2. Avec la commande :

```c
system("pause");
```

➜ La console affiche :

```nginx
Appuyez sur une touche pour continuer...
```

et attend que l'utilisateur appuie sur une touche.

Expliquer :

- system est une fonction standard du C (dans <stdlib.h>).
- "pause" est une commande Windows qui met le programme **en attente**.

Partie I : Programme “Bonjour” :

1. Compiler puis exécuter le projet ci-dessous. Que remarquez-vous ?

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    printf("Hello world !\nBonjour");
    return 0;
}
```

Réponse :

Le programme affiche :

```nginx
Hello world !
Bonjour
```

Justification :

- **printf** est utilisée pour afficher du texte à l'écran.
- Le caractère **\n** est un **saut de ligne**, il indique à l'ordinateur de passer à la ligne suivante avant d'afficher "Bonjour".

2. Enlevez le « ; » de la fin de la 5e ligne. Recompiler puis exécuter votre projet. Que remarquez-vous ?

Code modifié :

```c
printf("Hello world !\nBonjour")   // pas de point-virgule
```

Résultat :
Le compilateur affiche une **erreur de syntaxe**.

Justification :
En langage C, chaque **instruction** doit se terminer par un **point-virgule (;)**.

3. Remplacer \n par d'autres caractères d'échappement et observer les résultats ?

Caractère d'échappement	Observation	Justification
`\n`	Saut de ligne	Passe à la ligne suivante
`\t`	Tabulation horizontale	Ajoute un espace de tabulation (comme 4 à 8 espaces)
`\b`	Efface le caractère précédent	Le curseur revient d'un espace en arrière
`\r`	Retour au début de la ligne	Le texte qui suit écrase le début de la ligne
`\\`	Affiche le caractère `\`	Nécessaire car `\` est un caractère spécial
`\0`	Fin de chaîne	Tout caractère après `\0` n'est pas affiché
`\a`	Émet un bip sonore	Fait sonner le haut-parleur du système (si activé)

Exemple :

```c
printf("Hello\tworld");   // affiche "Hello    world"
printf("Hello\bworld");   // affiche "Hellworld"
printf("Hello\rworld");   // affiche "world"
```

Partie II : Exercices(Séances 2) :

1. Programme pour échanger les valeurs de deux variables

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int A, B, temp;

    printf("Entrez la valeur de A : ");
    scanf("%d", &A);
    printf("Entrez la valeur de B : ");
    scanf("%d", &B);

    // Échange des valeurs
    temp = A;
    A = B;
    B = temp;
```

```c
    printf("Après échange : A = %d, B = %d\n", A, B);
    system("pause");
    return 0;
}
```

Justification :
On utilise une **variable temporaire (temp)** pour stocker la valeur de A avant de l'écraser par B.
Ainsi, les valeurs de A et B sont échangées sans perte d'information.

2. Programme pour calculer la distance entre deux points (A et B) :

```c
#include <stdio.h>
#include <math.h>

int main()
{
    int XA, YA, XB, YB;
    double DIST;

    printf("Entrez les coordonnées du point A (XA, YA) : ");
    scanf("%d %d", &XA, &YA);
    printf("Entrez les coordonnées du point B (XB, YB) : ");
    scanf("%d %d", &XB, &YB);

    DIST = sqrt(pow(XB - XA, 2) + pow(YB - YA, 2));
```

```c
    printf("La distance entre A et B est : %.2f\n", DIST);
    system("pause");
    return 0;
}
```

Justification :

On applique la **formule mathématique de la distance** entre deux points :

$$D = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}$$

La fonction **sqrt() (racine carrée)** et **pow() (puissance)** sont définies dans la **bibliothèque <math.h>.**